

LAB #3: SECURITY AND PRIVACY ISSUES IN ML

Helen Huy, McCay Rudick, Jess Alencaster

Professor Fang

Due Date: April 20th, 2026

1. Security and Privacy Issues in ML-Based Systems

Privacy Issue: Membership Inference Attacks (MIA)

Link: Comprehensive Privacy Analysis of Deep Learning [2]

Description: Machine learning models inadvertently memorize training data through the Stochastic Gradient Descent (SGD) algorithm. Thus, an adversary can analyze gradients to determine if a specific person's data was used to train the model [2]. This works remarkably well even on well-generalized models; for example, DenseNet leaked 74.3% accuracy under white-box conditions.

Active inference attacks (subvariant of MIA): In federated learning (FL), a malicious participant or parameter server can deliberately manipulate gradient updates (e.g., via gradient ascent). This forces the model to expose other participants' private data, achieving up to 92.1% attack accuracy on AlexNet and 87.3% on DenseNet [2], well above the 50% random-guess baseline.

Privacy Issue	Attack Type	AlexNet	DenseNet
1) MIAs (Passive)	White-box (gradients)	75.1%	74.3%
2) Active Inference Attacks (in FL)	Malicious aggregator (ascent + isolation)	92.1%	87.3%

Table 1: MIA Attack Accuracies [2]

How are MIA attacks launched?

ML models trained with SGD leave a footprint in their parameters for every training point, turning the model itself into a privacy leak. Even after training concludes, an adversary can run a target data point through the model and compute the gradient $g = \nabla_W L$. They then feed g , the model output $f(x)$, and label y into a pre-trained attack model h . If $\|g\| \approx 0$, the point was likely a training member [2].

Algorithm 1 White-box Membership Inference Attack [2]

- 1: **Input:** model f , point x , label y
 - 2: **Output:** $\hat{m} \in \{0, 1\}$ (1 = member, 0 = non-member)
 - 3: $g \leftarrow \nabla_W \mathcal{L}(f(x; W), y)$ (backpropagate)
 - 4: **return** $h(g, f(x), y)$ ($\|g\| \approx 0 \Rightarrow$ member)
-

Security Issue: Data Poisoning Attacks

Link: Poisoning Attacks on FL-based IoT IDS [3]

Description: Data poisoning attacks corrupt an ML model by injecting malicious data into the training process. In an FL-based Intrusion Detection System (IDS), this manifests as a backdoor attack: attackers gradually inject malicious traffic into training data via compromised IoT devices, blinding the detector so it misclassifies malicious traffic as benign [3].

How are data poisoning attacks launched?

Data poisoning is a strategic execution rather than a single algorithm:

1. The attacker compromises IoT devices across multiple local networks (no gateway compromise needed).
2. They inject malicious traffic into the benign training stream at a controlled Poisoned Data Rate (PDR).
3. The Security Gateway unknowingly trains on poisoned data, backdooring its local model.
4. Poisoned local models are sent to the aggregator and averaged into the global model.
5. The corrupted global model now misclassifies malicious traffic as benign across all gateways.

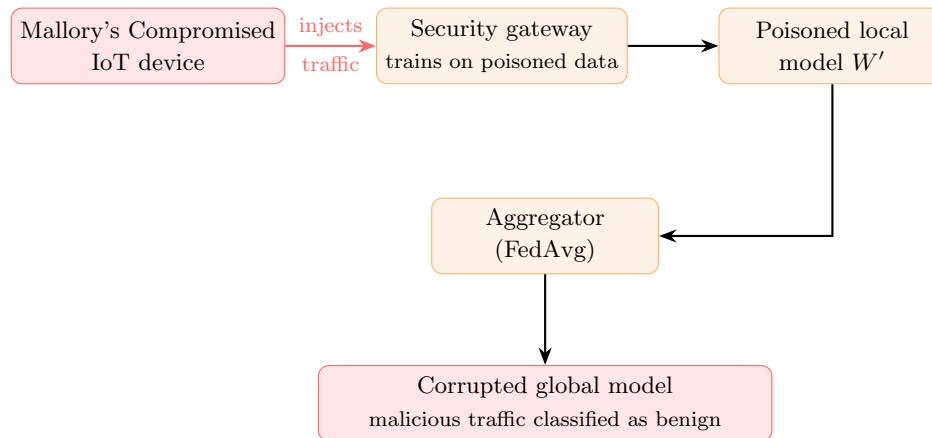


Figure 1: Backdoor Data Poisoning Attack Flow, adapted from Nguyen et al. (2020) [3]

Why are data poisoning attacks dangerous?

By design, they are stealthy. Even after the global model is backdoored, it continues to classify normal traffic correctly, giving operators no indication of compromise. Even when differential privacy defenses are applied, the attack still achieves 100% backdoor accuracy, while only dropping normal traffic detection to 94.2% [3]. Furthermore, this specific poisoning attack, unlike conventional poisoning attacks, only requires control of IoT devices, which are inherently vulnerable due to their limited computing capability, minimal built-in security, and low-cost hardware, where 50% of commercially available IoT devices contain significant security flaws [4]. Thus, in the context of IoT devices, this attack is far more practical to execute in real-world deployments compared to conventional poisoning approaches.

2. Solutions and Applicable Platforms

Solutions to Identified Vulnerabilities

- **Solution for MIA (Differential Privacy):** To mitigate Membership Inference Attacks (MIA), Differential Privacy (DP) can be integrated into the Stochastic Gradient Descent (SGD) algorithm [1]. By mathematically clipping the per-sample gradient to bound its sensitivity, then injecting calibrated Gaussian noise into the updates, the system obscures the footprint of any single data point. This makes it mathematically improbable for an attacker to conclude if $\|g\| \approx 0$, meaning an attacker cannot be certain that a specific user's information was part of the training set.
- **Solution for Data Poisoning (Federated Learning + Advanced Aggregation):** Raw data exposure is greatly reduced with the introduction of Federated Learning (FL). However, the aggregation phase is still vulnerable to poisoned local models. To solve this, an advanced aggregation method must be used (such as Trimmed Mean [6]) rather than standard FedAvg. This approach helps detect and filter statistical outliers (the poisoned gradients) before they can be merged into the global IDS.

Applicable Platform

Platform: TensorFlow Privacy (TensorFlow Privacy Guide).

TensorFlow Privacy is an open-source library built on TensorFlow. It can implement DP-SGD, the exact mechanism described in our MIA solution above. It provides built-in gradient clipping, Gaussian noise injection, and the “moments accountant” that tracks the cumulative privacy budget (ϵ , δ) across training steps [1].

Using this platform, we can demonstrate MIA vulnerability through the following steps:

1. Train a Convolutional Neural Network (CNN) without DP to expose gradient leakage.
2. Apply DP-SGD with varying ϵ values to represent different privacy budgets.
3. Observe how increasing noise reduces MIA attack accuracy at the cost of model utility.

All of these steps allow us to practically study the privacy-utility tradeoff.

3. Modeling the System: Membership Inference Attacks

System Model

We extend the Federated Learning-based IoT IDS proposed by Nguyen et al. (2020) [3] by replacing their GRU-based local model with a 1D-CNN architecture as implemented by Torre et al. (2025) [5]. In this architecture, distributed Security Gateways collaborate with a central Aggregator to train a global anomaly detection model without ever exposing the raw IoT traffic data.

Specifically, the block diagram of this architecture and its attack surface is illustrated in Figure 2:

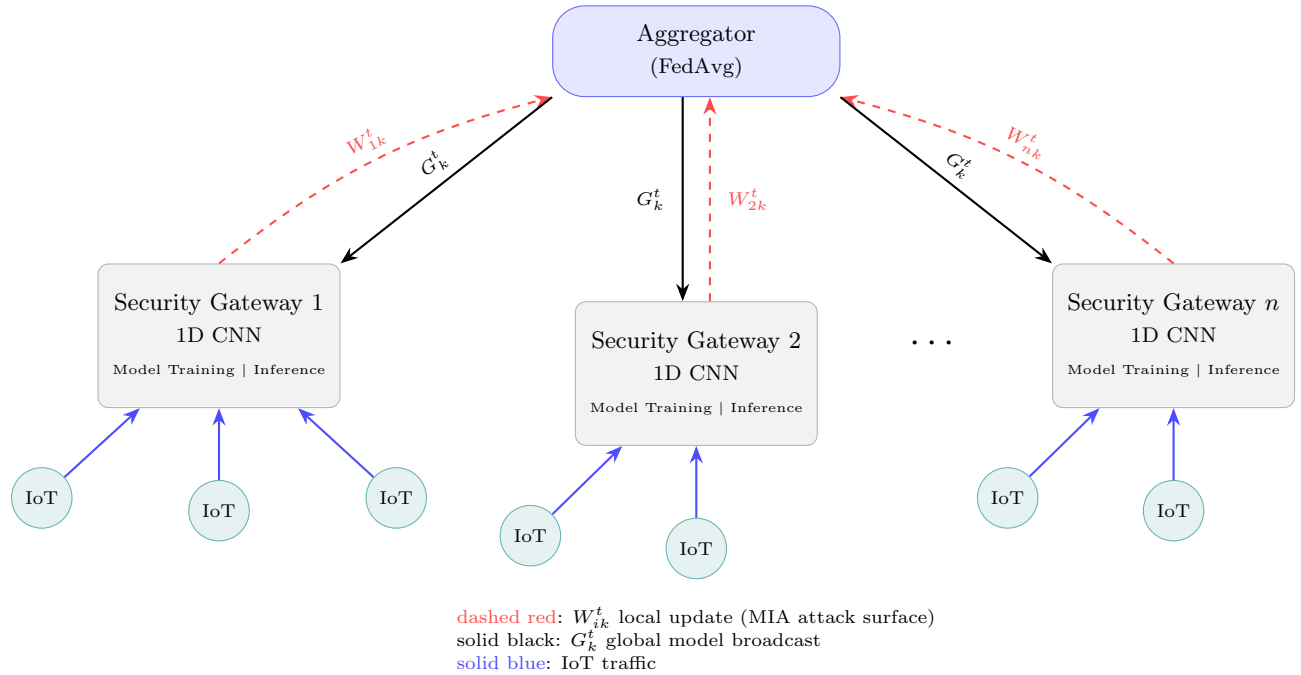


Figure 2: System Model for FL-based IoT IDS under MIA, combining architectures from Nguyen et al. (2020) [3] and Torre et al. (2025) [5].

Trust Model

We then consider the honest-but-curious aggregator threat scenario. We adopt the standard FL assumption that the aggregator correctly follows aggregation protocols to maintain the global model, but may passively exploit the local model updates it receives to infer private membership information about individual training data points [2]. The trust relationships between system components are summarized below in Table 2.

Entity	Trusted By	Trust Level	Rationale
IoT devices	Security Gateway	Full	IoT devices connect to their local gateway by design with no adversarial competition.
Security Gateway	Aggregator	Partial	Gateways must submit updates but cannot verify the aggregator is not curious.
Aggregator	W_{ik}^t (local updates)	Untrusted	Curious aggregator exploits gradient updates to infer private membership [2].
Security Gateway	G_k^t (global model)	Full	Gateways accept AND deploy the global model to perform local inference.
Security Gateway	Security Gateway	None	Participants cannot observe each other's updates. Only the aggregated global model is visible.

Table 2: Trust relationships in federated intrusion detection setting.

Threat Model

- **Adversary Goal:** The adversary aims to determine whether a specific IoT traffic record, say x , was included in some private training dataset D_{ik} of a target Security Gateway i . Formally, the adversary seeks to compute the membership prediction $\hat{m} \in \{0, 1\}$, where $\hat{m} = 1$ indicates that $x \in D_{ik}$ and $\hat{m} = 0$ indicates otherwise. In this IoT IDS context, this allows the adversary to infer sensitive behavioral information about users whose devices generated the traffic.
- **Adversary Capability:** This adversary is an honest-but-curious aggregator that correctly executes FedAvg, but passively observes all local model updates W_{ik}^t uploaded by Security Gateways across some T training rounds. The adversary does not disrupt the protocol, compromise any Security Gateway, or inject malicious updates.
- **Adversary Knowledge:** The adversary operates within a white-box setting. Therefore, it has full access to the 1D CNN architecture, model parameters, and the per-layer gradient vectors $g = \nabla_W \mathcal{L}(f(x; W), y)$, computed from the uploaded local updates. Importantly, this gradient access is the key enabler of MIA, as later CNN layers retain some distinguishable gradient “footprints” for training membership versus non-members [2].
- **Attack Vector:** The attack vector is the gradient upload channel. Specifically, this consists of the local model updates W_{ik}^t uploaded by each Security Gateway to the Aggregator during the training phase. As we illustrate in Figure 1, SGD forces $\|g\|$ toward zero for training members. This creates a statistically distinguishable signal that the adversary can exploit via some pre-trained attack model, say $h(g, f(x), y)$.
- **Assumptions:** We assume all Security Gateways are honest. Further, we assume the adversary controls no Security Gateways and does not modify the aggregation protocol. The adversary’s advantage here comes solely from passive observation of gradient uploads over multiple training rounds.

References

- [1] Martín Abadi et al. “Deep learning with differential privacy”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 308–318.
- [2] Milad Nasr, Reza Shokri, and Amir Houmansadr. “Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning”. In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019, pp. 739–753. DOI: 10.1109/SP.2019.00065. URL: <https://ieeexplore.ieee.org/abstract/document/8835245>.
- [3] Thien Duc Nguyen, Phillip Rieger, Huili Yalame, et al. “Poisoning Attacks on Federated Learning-based IoT Intrusion Detection Systems”. In: *Proceedings of the Workshop on Decentralized IoT Systems and Security (DISS)*. 2020. URL: <https://www.ndss-symposium.org/wp-content/uploads/2020/04/diss2020-23003-paper.pdf>.
- [4] P. K. Sadhu et al. “Prospect of Internet of Medical Things: A Review on Security Requirements and Solutions”. In: *Sensors* 22.19 (2022), p. 7433. DOI: 10.3390/s22197433. URL: <https://www.mdpi.com/1424-8220/22/19/7433>.
- [5] D. Torre et al. “Toward Enhancing Privacy Preservation of a Federated Learning CNN Intrusion Detection System in IoT: Method and Empirical Study”. In: *ACM Digital Library* (2025). URL: <https://dl.acm.org/doi/epdf/10.1145/3695998>.
- [6] Dong Yin et al. “Byzantine-robust distributed learning: Towards optimal statistical rates”. In: *International Conference on Machine Learning (ICML)*. PMLR. 2018, pp. 5650–5659. URL: <https://arxiv.org/abs/1803.01498>.